

庞玉博

☎ 15331874278 ✉ 3199608900@qq.com

📅 24岁 👤 男 📅 2年经验



教育背景

黑龙江大学

物联网工程 (本科)

2021.09 - 2025.07

专业技能

• 编程能力与开发工具

- 熟练掌握Python编程，具备良好编码规范
- 熟练使用Git进行版本控制，掌握GitHub协作流程
- 熟练使用Claude、CodeX等工具辅助代码生成、调试与架构设计

• RAG与数据工程

- 熟悉LangChain框架，熟悉检索增强生成(RAG)原理与实现，掌握文档切分、Embedding生成及混合检索
- 熟练使用Milvus、ChromaDB等主流向量数据库
- 熟悉neo4j图谱的实体与关系抽取，掌握构建图谱的流程
- 熟练使用Numpy、Pandas等工具进行高效的数据处理与分析

• Agent与应用开发

- 熟练使用LangGraph框架，具备智能体工作流编排经验
- 熟悉低代码平台Dify、Coze，可用于快速搭建Agent测试原型
- 具备Agent Skill的模块化封装能力，能通过自定义Skill与RAG技术大幅扩展Agent的知识与能力边界。

• 模型应用实现和部署

- 熟悉基于VLLM的大模型端部署方案
- 熟练掌握前后端一体化开发流程，具备 AI 应用系统架构设计与工程落地能力，能够独立完成从需求分析、系统设计到功能开发与部署上线。
- 具备 AI 产品完整生命周期实践经验，熟悉从业务调研、需求分析、方案设计、技术开发到产品运营优化的完整流程。

工作经历

海纳智联数字科技

大模型应用开发工程师

2024.10 - 2026.08

项目经历

【多 Agent 智能设备售后运维中台】

2026.02 - 2026.06

技术栈：

Python · FastAPI · LangGraph · Vue3 · TypeScript · Redis Stream · PostgreSQL · Milvus · Docker ·
LangChain · DashScope · MCP

项目背景：

传统开水器售后运维模式下，设备上报海量遥测数据，依赖人工盯盘识别故障，故障发现滞后；故障发生后缺少标准化故障诊断流程，售后工单生成效率低，安全类故障存在处置风险。同时海量设备遥测数据如果直接送入大模型会带来成本与性能压力，针对以上问题，设计实现告警驱动的 Supervisor 多专家 Agent 运维中台，完成故障诊断、安全校验、工单生成、分级 HITL 人工审核全闭环售后能力。

项目描述：

面向商用开水器售后垂直运维场景的 AI 运维中台。系统以设备告警作为智能体触发源，规避海量遥测直接调用大模型；基于 LangGraph 实现 Supervisor 多专家 Multi-Agent 协作，包含诊断 Agent、安全策略 Agent、工单分派 Agent；结合 Milvus 向量库构建维修手册 RAG 知识库，输出带溯源引用的故障诊断结论；使用 PostgreSQL 完成多租户业务数据存储，Redis Stream 实现告警任务异步解耦；通过分级 HITL 人工审核机制管控高风险故障工单质量，实现设备遥测接入、告警规则、AI 多专家诊断、工单审批、设备模拟器完整业务闭环。

项目职责：

- 基于 LangGraph 设计实现 Supervisor Multi-Agent 多智能体编排架构，摒弃固定流水线伪多智能体方案。

负责智能体架构设计与编码落地，不写死节点执行顺序；Supervisor 调度 Agent 依据告警等级、故障码、遥测快照上下文动态编排子任务，支持按需选取子 Agent、跳过非必要逻辑、并行执行专家任务、触发重试逻辑，拆分 Diagnosis 故障诊断、Safety 安全校验、Triage 工单分派三个独立子 Agent。

依托 LangGraph Graph State 全局状态统一托管全链路上下文，保存原始告警快照、各 Agent 结构化输出、agent_trace 完整执行日志，依靠 State 完成 Agent 间上下文传递，不使用中间件作为 Agent 通信总线。实现多 Agent 输出冲突仲裁，针对结果矛盾做合并、重调用或强制升级人工审核；调度决策、输入输出、仲裁理由全部存入 agent_trace，用于问题排查、链路溯源以及数据集沉淀。

- 设计落地分级 HITL 人在回路审核机制，搭建工单安全兜底能力。

完成 HITL 规则编码，定义四级告警审核策略：P0 安全故障强制人工审核，P1 默认人工审核，P2/P3 低风险故障可自动生成工单。Safety Agent 输出最高优先级 force_hitl 标记，标记生效时工单强制进入人工待审队列，不受派单 Agent 输出影响。针对 LLM 的 Schema 校验失败、大模型服务异常、RAG 检索为空等降级场景，强制升级人工审核，阻断错误 AI 结果直接派单；对接前端审批接口，支持坐席通过、驳回工单，留存审核记录，闭环工单状态，规避大模型幻觉带来的售后风险。

- 搭建维修手册专属 RAG 检索链路，实现可溯源故障资料检索，约束大模型禁止编造维修流程。

维修手册原始文件存入 OSS 对象存储；文档解析切块后，文本元数据存入 PostgreSQL，向量存入 Milvus，文档块维护 chunk_id、doc_id、章节元信息。将手册检索封装为 Function-Calling 工具，仅对 Diagnosis Agent 开放；大模型只能基于检索片段输出诊断，无检索结果时禁止编造内容，诊断输出携带 citation 引用，关联原始文档与章节编号，实现诊断结论完整溯源。

- 设计设备宽表遥测接入体系，落地两套鉴权隔离，高性能时序写入，严格解耦上报与大模型流程。

定义设备上报宽表契约，单条上报携带全量传感器字段，优化时序写入压力；基于 FastAPI 搭建设备接入接口，设备端使用 X-Device-Token 并与 SN 码强绑定鉴权，运维人员采用 JWT+RBAC，划分 admin/operator/viewer 角色，实现人机鉴权隔离，规避越权风险。采用内存缓冲 + 后台批量写入，上报接口快速响应，不在 HTTP 链路执行数据库重写入与大模型调用；后台异步批量写入 PG 分区时序表，明细热数据留存 14 天，定时任务生成小时、日粒度聚合表保障历史指标可查询。上报链路仅处理鉴权、缓冲、落库、轻量阈值判断，不执行任何 LLM/Agent 逻辑；仅阈值命中产生告警后，将告警 ID 推送 Redis Stream，由独立 Worker 异步消费再执行多智能体流程，实现流量削峰与故障隔离。

- **基于 Redis Stream 构建高可靠告警任务队列，实现消息可靠处理与 Agent 背压保护。**

使用消费组支持多 Worker 实例，完整实现消息生命周期：XADD 入队、处理成功 XACK 确认、失败消息进入 pending 列表、超时消息 XCLAIM 重试、超最大重试次数转入死信队列并更新告警状态，保障任务不丢失。增加 Agent 层背压保护，限制 Supervisor 全局并发数；队列积压时优先处理 P0/P1 高优先级告警，P2/P3 低优先级任务做降级延迟处理，保障安全故障优先处置；对外暴露 lag、pending 等监控指标观测队列水位。

- **落地多租户安全隔离与模型降级兜底。**

所有业务表增加 tenant_id 租户标识，接口强制过滤租户 ID，防范 IDOR 跨租户漏洞；通过 Pydantic 定义业务模型与 Agent 输出 Schema，对大模型输出做强类型校验；Milvus、大模型网关故障时启用模板降级，强制流转人工审核，保障系统可用。

项目成果：

- 构建告警驱动 Supervisor 多专家 Agent 系统，实现故障诊断-安全校验-工单生成完整售后闭环，通过 agent_trace 完整留存智能体执行轨迹，便于调试审计与项目演示；单告警智能体完整链路平均处理耗时控制在 **1.8-3s**，支持单机同时并发处理 **80+** 告警任务。
- 搭建维修手册 RAG 知识库，实现诊断结果 citation 溯源，有效抑制大模型幻觉，故障诊断可直接引用原始维修手册资料；手册检索召回准确率达到 **87%**，AI 自动工单有效率提升 **42%**。
- 完成多租户 RBAC 权限体系，PG 分区 + Redis Stream 架构实现海量设备遥测削峰解耦，做到仅告警事件进入大模型；过滤 95% 普通遥测数据不调用大模型，**LLM 调用成本下降约 75%**，可支撑上万台设备并发上报。
- 实现完整 HITL 人审管控，漏水、极低水位等高风险故障 100% 强制人工复核，规避 AI 错误派单带来的现场安全风险；低风险故障自动派单占比 **60%**，售后工单人工处理工作量显著下降。
- 交付可演示完整 MVP，包含模拟器、后端服务、运维前端，覆盖漏水、超温、耗材提醒等多类故障验收场景；模拟器支持模拟 **1000+** 台虚拟设备压测，可直接用于项目答辩与原型验证。

【基于 Agentic-RAG 的企业智能知识协同中枢平台】

2025.09 - 2025.12

技术栈

Python、FastAPI、Langchain、LangGraph、MinerU、Vue3、ElementPlus、TypeScript、vLLM、Neo4j、Milvus、Elasticsearch、PostgreSQL、Docker、Docker-Compose、Redis、MinIO

项目背景

企业内部沉淀大量规章制度、技术文档、运维资料，传统文档检索方式仅依靠关键词搜索，查询效率低下，容易出现信息不全；同时普通 RAG 方案容易产生幻觉，无法处理敏感文档权限隔离、人工审批、多跳关系查询等企业刚需。为解决文档权限管控、检索幻觉、知识关联分析等痛点，搭建一套具备租户隔离、权限下推、图谱推理、人机协同审批能力的 Agentic-RAG 企业知识中台，面向内部员工提供安全可控的智能问答服务。

项目描述

一款面向企业内部使用的 To-B 知识中台系统，后端基于 Python+FastAPI 开发网关服务，底层通过 vLLM 实现本地大模型部署；基于 LangChain 封装检索工具，LangGraph 编排 Agentic-RAG 智能问答工作流。平台支持多租户知识库管理、PDF/Word/TXT 多格式文档上传解析、文档切片入库、Milvus 向量库 + ES 关键词混合检索、Neo4j 知识图谱抽取与多跳推理；依托 LangGraph 实现问题改写、证据校验、检索循环重试、敏感问题 HITL 人工审批等复杂流程；前端采用 Vue3 + TypeScript 开发问答页面、知识库管理、图谱可视化、审批后台；使用 Docker+Docker-Compose 完成整套服务容器化部署；全程采用权限条件下推的安全方案，禁止先召回再过滤，从根源杜绝越权访问、敏感文档泄露风险。

项目职责：

- **后端架构与多模型网关设计。**

使用 Python+FastAPI 搭建高可用知识中台后端，实现接口拦截、JWT 鉴权、全局异常捕获以及全链路日志埋点。基于适配器 + 工厂模式自研模型网关，统一封装对话、Embedding、Rerank 模型能力，兼容 vLLM 私有化模型与云上大模型，支持环境变量动态切换，完成模型调用限流、熔断降级、Token 统计与异常告警。基于 LangChain 封装检索、图谱查询工具，借助 LangGraph 搭建 Agentic-RAG 智能体工作流，依靠 State 管理对话状态，搭配 Redis Checkpoint 实现状态快照持久化，完成多意图路由、检索重试、HITL 人工审批、断点续跑，解决传统线性 RAG 无法适配企业复杂业务流程的缺陷。

- **多层权限架构与数据安全落地。**

面向 ToB 多租户场景，设计 RBAC 角色权限 + 知识库 ACL + 文档密级三层权限体系，精细化管控不同角色的功能、知识库、涉密文档访问权限。落地权限下推安全策略，舍弃先召回再过滤的风险方案，将租户 ID、可读知识库、文档密级条件直接下推至 ES、Milvus、Neo4j 底层检索过滤，数据源层面拦截越权数据，达成多租户强隔离，操作全程可溯源审计，满足企业数据合规需求。

- **智能文档解析与异步检索流水线优化。**

基于 Redis Worker 构建异步任务流水线，解耦文档上传、存储、解析、切片入库、图谱构建等耗时任务，避免接口阻塞，支持批量文档处理。采用 MiniIO 完成文档对象存储；借助 MinerU 清洗页眉、水印等无效内容，依据文档标题层级做语义切片，改善固定切片语义断裂问题。搭建 ES+Milvus 混合检索，使用 RRF 算法融合多路检索结果，配合 Rerank 重排优化上下文质量，提升检索召回效果，缓解大模型幻觉。

- **知识图谱开发与脏数据治理。**

采用手动触发式图谱抽取，仅对核心业务文档执行图谱任务，减少无效算力消耗与脏数据。开发异步抽取任务提取实体、业务三元组，通过同义词归一化、实体合并做数据清洗，使用 Neo4j MERGE 语法幂等写入，规避实体重复、关系错乱。依托 Neo4j 实现多跳关系推理，支撑组织架构、业务链路、岗位权责这类复杂关联问答，补足传统 RAG 缺少关系推理的短板。

- **流式交互与前后端协同。**

基于 SSE 流式协议实现实时问答，支持断线重连，向前端推送流式回答、文档溯源、智能体节点日志、审批状态、异常事件。前端采用 Vue3+TypeScript+ElementPlus 开发，覆盖智能问答、知识库管理、文档运维、图谱可视化、HITL 审批、日志审计模块，实现权限自适应渲染、业务状态联动，完成整套业务闭环。

- **容器化部署与稳定性保障。**

通过 Docker、Docker-Compose 对后端及中间件全组件容器编排，统一开发、测试、生产环境配置，消除环境与依赖差异。配置容器内存限制、健康检查、数据卷持久挂载；引入分布式锁、任务幂等、失败重试、死信队列，处理异步任务并发冲突、重复执行、任务堆积，保障系统高并发高可用。

项目成果

- 落地权限下推检索方案，彻底规避先全量召回后过滤带来的越权泄露风险，满足企业多租户、文档密级隔离与安全审计要求。
- 基于 LangGraph 构建 Agentic-RAG 工作流，新增检索重试、输出校验守卫节点，显著降低大模型幻觉编造现象，知识库问答准确率大幅提升。

- ES+Milvus 混合检索 + 重排召回方案，专有名词、内部文档检索效果优于纯向量检索；检索召回指标 Recall@10 $\geq$ 91%。
- 完成 Neo4j 企业知识图谱模块开发，实现文档实体三元组抽取、多跳关系推理查询，知识关联查询命中率达到 86% 以上。
- 整套平台 Docker 容器化部署，异步 Worker 任务流水线稳定处理文档切片、图谱抽取、文档删除等任务；Agent 断点人工审批功能上线，复杂问答流程稳定性显著提升。

【轻量化 RAG 内部文档问答原型】

2024.11 - 2025.01

技术栈

Python • FastAPI • LangChain • ChromaDB • MinerU • Docker

项目背景

公司内部沉淀大量产品手册、接口开发文档，团队查找资料主要依靠本地翻阅，文档检索效率低下。为快速验证 RAG 技术落地可行性，搭建一套轻量化问答原型用于团队内部试用，同时完成文档解析、切片、检索链路的技术预研，为后续企业知识中台项目沉淀选型依据。

项目描述

面向团队内部使用的轻量化 RAG 问答原型系统，实现 PDF 文档上传解析、文本切片向量化、向量检索、流式智能问答完整链路。项目聚焦基础 RAG 能力验证，不实现多租户、知识图谱、复杂权限管控等企业级特性，仅服务内部小范围验证使用，通过 Docker 容器化快速部署。

项目职责

基于 FastAPI 搭建后端服务，封装文档上传、问答查询接口，实现 SSE 流式问答返回，完成接口异常捕获与基础参数校验。引入 MinerU 处理 PDF 文档，清洗页眉、水印等无效噪声，基于文档标题层级实现语义切片，规避固定长度切片带来的语义断裂问题。基于 LangChain 搭建基础 RAG 业务链路，使用 ChromaDB 完成向量存储与检索；增加输出校验逻辑，约束大模型仅基于检索到的文档片段生成回答，抑制无依据编造。开展对比实验，测试不同 Embedding 模型、切片窗口大小对检索召回效果的影响，输出技术选型评估报告。使用 Docker 完成项目容器化打包，屏蔽环境依赖问题，方便团队成员快速部署体验。

项目成果

完成轻量化 RAG 问答原型，支持 PDF 文档解析与智能问答，内部小范围试用，文档问答准确率 78%。验证 MinerU 文档解析、语义切片方案的可行性，明确 ChromaDB 原型适配、生产环境需替换 Milvus 的选型结论，预研得到的调参经验直接复用至后续企业智能知识协同中枢平台，减少正式项目踩坑成本。

自我评价

- 对AI技术充满热情，始终保持对前沿技术的关注。具备极强的逻辑思维与抗压能力，善于攻克技术难关，致力于通过技术创新解决复杂的业务痛点。
- 具有极强的责任心，性格和善，可以快速融入团队，良好的沟通能力和团队合作意识。
- 有较强的抗压能力，遇到问题能够沉着冷静地应对，做到具体问题具体分析，积极寻求解决方案。